WEEK 5 · DAY 3 · INSTRUCTOR NOTES

Regression Forecasts + Honest Ranges

For the trainer only.

The one idea for today: *a forecast is only honest if (1) every feature was known before the week started, (2) it is scored against the best baseline on the same weeks with the same information, and (3) its range comes from real past misses and has been checked on weeks it never saw.*

At a glance

Length	~2 hours, including 20 minutes for final-project Checkpoint 4
Participants leave able to	build lag and calendar features without leaking the future; fit and sanity-check a `LinearRegression` forecast; compare forecasts like with like; build a 90% band from honest past misses; check a band's coverage; say a forecast with its range
Covers (original)	Week 6 Code Book, *Topic 6.2 Regression-based forecasting* and *Topic 6.3 Uncertainty: prediction intervals*; Week 6 quiz topics *regression-based forecasting* and *uncertainty and prediction intervals* (Quiz 4, questions 7–18, sat on Week 6 · Day 2)
Dataset	`week-06/data/ops_tickets_weekly.csv` (loaded in the notebook): 104 weeks, 1 Jan 2024 – 22 Dec 2025
Files you need open	animated slides, your **solution** notebook, the rule book PDF, the final-project checkpoint table

The day before: checklist

☐ Run the **solution** notebook top to bottom: 14 of 14 ✅ (about 10 seconds).

☐ Re-read the end of W5D2: the **rate seasonal naive** (last year's tickets per terminal × today's terminals) scored **2.24** on the 8 held-out weeks. Today's regression is judged against that.

☐ Remind participants to bring their final-project **honest test** (Checkpoint 4): a split by time and a baseline comparison table, on **their** data, on NIGCOMSAT systems.

Run of show (2 hours)

Time	Block	What happens	Slides
0:00	**Hook**	Tobi's "I'm on my way" vs Amaka's "30 to 90 minutes, Third Mainland permitting"	1–3
0:08	**The bar**	Last time's scoreboard: rate seasonal naive 2.24	4

Time	Block	What happens	Slides
0:10	**Part 1: the regression**	Leak rule, five columns, the clock face, 104 → 52 rows, coefficients	5–10
0:22	**Part 2: like with like**	"Half the error!", the head start, the like-with-like table	11–14
0:30	**Part 3: honest ranges**	5 of 8; why; the honest recipe; 7 of 8; the trap (2 of 8); the rate band (8 of 8); how to say it	15–23
0:40	**Hands-on**	Part A (12 min), B (15 min), C (18 min), D (5 min)	25
1:30	**Checkpoint 4**	one by one: honest test (see below)	24
1:50	**Wrap-up**	recap; tease backtesting	26

Explain it like this (in plain English)

- **A forecast is a regression with one rule.** A regression predicts one column (the *target*: this week's tickets) from other columns (the *features*). To be a forecast, every feature must be **known before the week starts**. That is the Week 4 Day 3 "14:00 test" again, for weekly data: *could I compute this on Monday morning?*
- **`shift(1)` vs `shift(-1)`.** `shift(1)` moves the column down one row, so each week's row shows **last** week's count: on 3 Nov you see 27 Oct's 74. `shift(-1)` moves it up, so each row shows **next** week's count (10 Nov's 54 on 3 Nov's row). That's reading tomorrow's newspaper: it would score beautifully and is impossible in the NOC.
- **The five columns.** `tickets_lag1` (last week), `tickets_lag52` (the same week last year), `terminals` (how many terminals are live this week; known from the rollout plan), and `sin52` + `cos52` (the week of the year).
- **Why sin and cos.** Week numbers run 1…52 and then jump back to 1. As a plain number, week 52 and week 1 look 51 apart, but they are neighbours. Put the weeks round a circle like a clock face: `sin52` and `cos52` are the across and up-down position of the week on that circle. Two columns describe the whole year, and the end joins the start. The catch, which matters later: a circle is **smooth**, and this data's rainy season is a **switch** (on in early April, off at the start of November).
- **Rows lost.** `shift(52)` has nothing to copy for the first 52 weeks, so `dropna()` removes them: 104 → **52** rows. 2024 is still used, as the source of every `lag52` value.
- **The split.** The same 8 held-out weeks as last time (3 Nov – 22 Dec 2025). Training = the 44 rows before them (30 Dec 2024 – 27 Oct 2025). Five columns learnt from 44 rows is a small exam.
- **What `LinearRegression` does.** It finds one weight per column (the *coefficient*) plus a starting number (the *intercept*) so that *weight* × *column* + *weight* × *column* + … + *intercept* is as close as possible to the target on the training weeks.
- **Reading the coefficients.** `terminals` comes out **+0.0231** per terminal: about **+23 tickets a week for every 1,000 extra terminals**. More network, more tickets. A **negative** sign would be a warning: something is broken, or two columns are fighting each other. `tickets_lag1` is +0.497 (about half of last week carries over), `tickets_lag52` +0.226, `sin52` +0.888 and `cos52` −2.938 (together they shape the yearly wave). The intercept (−22.1) has no meaning on its own.

- **"Half the error of naive!"** The regression's MAE is **5.85** on the 8 weeks; naive's was **11.25**. But naive stood on 27 Oct and said 74 for all 8 weeks, blind. The regression got last week's **real** count (`tickets_lag1`) every week: it's a **one-week-ahead** forecast. Give naive the same hint ("this week = last week's real count") and it scores **6.75**. The regression's honest edge is **0.9** of a ticket a week.
- **Beat the best baseline.** Last time the rate seasonal naive scored **2.24**, forecasting all 8 weeks blind. The regression, with a head start, still loses. That's a result, not a failure: the code book itself says a model "has to beat whichever baseline actually won".
- **A prediction interval (band).** A range the real value should fall inside a stated share of the time. "90% band" = right about 9 times in 10. **Coverage** = how many held-out weeks actually fell inside. With 8 test weeks, a real 90% band should catch 7 or 8.
- **The code book's band.** Forecast ± 1.645 × the spread (standard deviation) of the training misses (2.84) = **± 4.67**. 1.645 is the bell-curve number that leaves 5% in each tail, so ± 1.645 sd is a two-sided 90% band. It covers **5 of 8** weeks (62.5%). Too narrow.
- **Why too narrow.** (1) The training misses are measured on weeks the model was **fitted to**, so they're flattering: sd 2.84 in training vs **6.99** on the held-out weeks, where it also forecast on average **4.1 tickets too high**. (2) The season: on 3 Nov the rains end. Last week was 74, the model said **70.9**, reality was **53** (a miss of −17.9). A smooth sin/cos wave can't drop like a step.
- **The honest recipe.** Walk forward through the training weeks: at each week, fit on the weeks **before** it only, forecast that week, write down the miss. Starting at row 12 (24 Mar 2025, so the first fit has 12 rows for 6 numbers) gives **32** honest misses. The **90th-percentile** miss (90% of misses were smaller) is **8.20**. Band = forecast ± 8.20 → covers **7 of 8**. The forecast didn't change; only the width did.
- **The trap.** Run the same recipe from 19 May (summer only): 24 calm misses, band ± **3.33**, covers **2 of 8**. The tallest miss in the first version (+15.1 on 7 Apr) is the week the rainy season switched on. *A band is only as honest as the weeks you measured it on:* if you forecast season changes, measure on season changes.
- **The rate forecast's band.** `rate_past_errors` stands at each of the **37** earlier weeks that have a full year behind them, makes the same 8-week rate forecast, and records all 8 misses (296 misses). The 90th percentile is ± **7.01**: narrower than the regression's band, and it covers **8 of 8**.
- **Don't turn the knob.** Raising z until the test weeks fit is choosing the band by looking at the answers: expo (W5D1). The width must come from misses **before** the test.
- **How to say it.** "Week of 3 Nov: most likely around **56**, plausibly **49 to 63**; expect bigger misses at season changes." (Actual: 53.) Three parts: most likely, a range, the thing that could blow it up.

Slide talking points

#	Say this	Ask this
3	Everyone has a Tobi. The NOC planner needs Amaka.	"Who would you rather plan a shift around?"
4	This is the bar. Not the weakest bar: the lowest one.	"What did the 2.24 method know that the others didn't?" (the network grew)
6	Quiz 4 Q7–Q8. Point at 3 Nov: green arrow from the past is fine; red arrow from the future is a leak.	"Which arrow could you draw on Monday morning?"

#	Say this	Ask this
8	Quiz 4 Q10. Hour hand: 11 o'clock and 1 o'clock are close. Plant the doubt: smooth vs switch.	–
9	Quiz 4 Q9.	"Why don't we just delete 2024?" (it feeds lag52)
10	Quiz 4 Q12. A sign check is free and catches broken features.	"What would a negative terminals sign mean?"
12	Let them enjoy it…	"What did each one know when it made each forecast?"
13	Quiz 4 Q11: one-step naive = 6.75.	–
14	Beat the best, not the weakest.	"Would you deploy the regression?"
16	Quiz 4 Q13–Q15. 5 of 8 is 62.5%, not 90%.	"If a mechanic says 90% sure and is right 5 times in 8…?"
17	Quiz 4 Q16. Marked its own homework + the season switch.	–
19	The tall bars are spring, the season change.	"Which weeks would you measure on?"
21	Same recipe, different weeks, 7 vs 2.	–
23	Quiz 4 Q17–Q18.	"Say Amaka's sentence for your own project."
24	Checkpoint 4 template.	–

Answer key

Task	Answer	Common mistake → what to say
A1	`LAG = 1`	`-1` copies next week back: tomorrow's newspaper. The hint says so
A2	`rows_lost = len(tickets) - len(feat)` = **52**	Typing 1 (that's lag1); the year-ago lag needs a full year
A3	`H = 8` → 44 training weeks, 8 test weeks	–
B1	`TARGET = "target"` → MAE **5.85**	Writing `target` without quotes (NameError) or `"tickets_raised"` (that column isn't in `feat`: KeyError)
B2	`terminals_coef = coefs["terminals"]` = **+0.0231**	Typing the number by hand is accepted if it matches; ask them to read it from `coefs`
B3	`NAIVE_COL = "tickets_lag1"` → **6.75**	`"tickets_lag52"` gives the year-ago value (a different, much worse baseline)

Task	Answer	Common mistake → what to say
B4	`False`	Some say True because 5.85 < 6.75 and 11.25. Point at the 2.24 row: the best baseline
C1	`Z = 1.645` → ± 4.67, covers **5**	Writing `90` or `0.9` : z is the bell-curve multiplier, 1.645
C2	`Q = 0.9` → ± **8.20**, covers **7**	Writing `90` makes numpy refuse (quantile must be between 0 and 1)
C3	`SUMMER_START = "2025-05-19"` → ± **3.33**, covers **2**	–
C4	`ORIGIN = len(tickets) - H` = 96 → ± **7.01**, covers **8**	Writing 104: forecasts beyond the data. The check fails
C5	`False`	–
D1	`"rate seasonal naive"`	"regression": it has the lower MAE of the two one-week methods, but the rate forecast is lower still, 8 weeks ahead, with a band that held
🌶️	`band_week1 = (rate_fc[0] - half_rate, rate_fc[0] + half_rate)` → about (49.4, 63.4)	Giving a single number. It needs a pair (low, high)

🏁 Final project · Checkpoint 4: ⚖️ Honest test (20 min)

Matches `week-6/final-project/Final-Project-Requirements.pdf`, Stage 3. Go round one by one:

Check	Pass if
Split by time	an `assert` proves every training timestamp is before every test timestamp
Embargo	a gap at the split at least as long as their longest feature window (W5D1)
Same period, same scorer	their simple baseline (naive / seasonal naive / control chart / threshold) is scored on exactly the same test period with exactly the same scorer as their method
The table	one row per method: how far ahead it forecast, what information it used, the score; like the slide 24 template
Brief-specific	**forecasting (C, E):** a 90% band whose coverage they checked on held-out data · **detection (A, E):** event-level precision and recall with the scoring rules written down first, no accuracy figure · **degradation / capacity (B, D):** the crossing call made from more than one starting point
Honest verdict	if the simple method wins or ties, they say so and recommend it
Data rules	real data on NIGCOMSAT systems only (training server or work laptop, not Colab); nothing raw on GitHub; nothing restricted pasted into AI tools

A table that shows their method losing is a **pass** if it's honest. Anyone without labelled events agrees a substitute (maintenance log, ticket history) with their unit and says so. Next checkpoint: **Week 6 · Day 3: 🎤 demo day** (working tool, report, 5-minute demo).

Troubleshooting

Problem	Fix
`NameError: name 'make_features' …` / `'mae'` / `'coverage'`	They skipped the 🧰 toolbox cell
`AssertionError: ✍️ Finish A3 first` (or B1, C2…)	An earlier task is blank; go back up
`KeyError: 'tickets_raised'` in B1	`feat` calls the answer column `"target"`
`ValueError: Quantiles must be in the range [0, 1]` in C2	They typed `90`; it's `0.9`
B1 MAE slightly different from a neighbour's	Shouldn't happen (no randomness). Check they didn't change `X_COLS` or `H`
Everything is weird	**Runtime → Restart session and run all**
Fast finishers	🌶️ task, then: "Which week does the regression's honest band miss, and why?" (3 Nov: the rains end)

Notes on the original material

- **The numbers reproduce.** The code book's 6.2 and 6.3 numbers come out the same when you run them: regression MAE 5.85, one-step naive 6.75, residual sd 2.84, held-out error sd about 7, coverage 5 of 8. Quiz 4's questions on these days all match the data.
- **"About 6 of the 8."** The code book's 6.3 text says the band covers "about 6 of the 8 held-out weeks (roughly 60–65%)". It covers **5** of 8 (62.5%); the percentage is right, the count is not. The quiz (Q15) says 5 of 8, which is correct.
- **"z = 1.645 # ~90% interval, one-sided-flavoured".** That comment is wrong: 1.645 is exactly the two-sided 90% value (5% in each tail). If someone asks, the band itself is built correctly.
- **"Roughly half the error of the best baseline."** True against naive (5.85 vs 11.25), and the code book itself then shows why it's unfair (cell 18). It never compares with a baseline that uses the network size; we do (W5D2's rate seasonal naive, 2.24), and it wins.
- **Why the band fails.** The code book blames choppier held-out weeks. The bigger cause is visible in the misses: the first held-out week is the end of the rainy season (−17.9), a step that the smooth sin/cos wave can't make.